LAPORAN PRAKTIKUM

STRUKTUR DATA

"*QUEUE*"

disusun Oleh:

NIDA TSABITA ARIBA

2511532025

Dosen Pengampu:

Dr. WAHYUDI, S.T., M.T.

Asisten Praktikum:

JOVANTRI IMMANUEL GULO

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

2025

KATA PENGANTAR

Alhamdulillahi rabbil ‘alamin, segala puji dan syukur saya panjatkan ke hadirat Allah SWT atas limpahan rahmat, nikmat, dan hidayah-Nya sehingga saya dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan judul “*QUEUE*”.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Struktur Data. Laporan ini diharapkan dapat memperdalam wawasan saya mengenai penerapan konsep struktur data, khususnya *QUEUE* dalam bahasa pemrograman Java, serta implementasinya dalam pengelolaan data yang bersifat dinamis.

Saya menyampaikan terima kasih kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Algoritma dan Pemrograman, serta asisten praktikum yang telah memberikan bimbingan dan arahan selama pelaksanaan kegiatan praktikum. Ucapan terima kasih juga saya sampaikan kepada teman-teman praktikan yang telah memberikan bantuan, dukungan, serta kesempatan berdiskusi dalam menyelesaikan tugas praktikum ini.

Saya menyadari bahwa laporan ini masih memiliki banyak kekurangan dan keterbatasan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan laporan praktikum di masa mendatang. Meskipun demikian, saya berharap laporan ini dapat memberikan manfaat bagi pembaca.

DAFTAR ISI

KATA PENGANTAR ii
DAFTAR ISI iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan 1
1.3 Manfaat Praktikum 1
BAB II PEMBAHASAN 3
2.1 Program *QueueArray* 3
2.2 Program *QueueLinkedList* 11
2.3 Program *ReverseData* 14
2.4 Program *IterasiQueue* 16
BAB III KESIMPULAN 19
3.1 Kesimpulan 19
DAFTAR PUSTAKA 20

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pembelajaran pemrograman Java, mahasiswa tidak hanya mempelajari dasar-dasar seperti perulangan dan percabangan, tetapi juga mempelajari struktur data untuk mengelola data secara efisien. Salah satu struktur data yang penting adalah *Queue* (antrian).

Queue merupakan struktur data yang menggunakan prinsip *FIFO* (*First In First Out*), yaitu data yang pertama masuk akan menjadi data pertama yang keluar. Struktur ini banyak digunakan dalam kehidupan sehari-hari, seperti antrian di loket pelayanan.

Pada praktikum ini, *Queue* diimplementasikan menggunakan *array* dalam bahasa Java. Program yang dibuat mampu melakukan operasi dasar seperti menambah data ke antrian (*enqueue*), menghapus data dari antrian (*dequeue*), serta menampilkan isi antrian.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep *Queue* serta mampu mengimplementasikannya dalam program sederhana.

1.2 Tujuan

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar *Queue* (antrian) dalam struktur data.
2. Mengetahui prinsip kerja *FIFO* (*First In First Out*).
3. Mempelajari implementasi *Queue* menggunakan *array* di Java.
4. Mengimplementasikan operasi dasar *Queue* seperti *enqueue*, *dequeue*, dan *display*.
5. Melatih kemampuan dalam membuat program pengelolaan data berbasis antrian.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini adalah:

1. Menambah pemahaman tentang struktur data *Queue*.
2. Membantu memahami konsep antrian dalam pemrograman.
3. Melatih kemampuan membuat program menggunakan *array* sebagai penyimpanan data.
4. Membiasakan penggunaan operasi dasar *Queue*
5. dalam pemrograman.
6. Meningkatkan kemampuan logika dalam menyelesaikan permasalahan berbasis antrian.

BAB II

PEMBAHASAN

2.1 Program *QueueArray*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan4_2511532025;

public class QueueArray_2511532025 {
int front_2025, rear_2025, size_2025;
int capacity_2025;
int array_2025[];

public QueueArray_2511532025(int capacity_2025) {
    this.capacity_2025 = capacity_2025;
    front_2025 = this.size_2025 = 0;
    rear_2025 = capacity_2025 -1;
    array_2025 = new int[this.capacity_2025];
}

boolean isFull(QueueArray_2511532025 queue) {
    return (queue.size_2025 == queue.capacity_2025);
}

boolean isEmpty(QueueArray_2511532025 queue) {
    return (queue.size_2025 == 0);
}

void enqueue_2025(int item) {
    if (isFull(this))
            return;
    this.rear_2025 = (this.rear_2025 + 1) % this.capacity_2025;
    this.array_2025[this.rear_2025] = item;
    this.size_2025 = this.size_2025 + 1;
    System.out.println(item + " enqueued to queue");
}

int dequeue_2025() {
    if (isEmpty(this))
            return Integer.MIN_VALUE;
    int item = this.array_2025[this.front_2025];
    this.front_2025 = (this.front_2025 + 1) %
this.capacity_2025;
    this.size_2025 = this.size_2025 - 1;
    return item;
    }

    int front_2025() {
            if (isEmpty(this))
                    return Integer.MIN_VALUE;
            return this.array_2025[this.front_2025];
    }

    int rear_2025() {
```

```
        if (isEmpty(this))
                return Integer.MIN_VALUE;
        return this.array_2025[this.rear_2025];
    }

    //mencetak elemen antrian
    void display_2025() {
        int i_2025;
        if (front_2025 == rear_2025) {
                System.out.printf("\nAntrian Kosong\n");
                return;
        }

        //kunjungi dari belakang dan cetak
        for (i_2025 = front_2025; i_2025 < rear_2025;
i_2025++) {
                System.out.printf("%d <-- ",
array_2025[i_2025]);
    }

    return;

}
}
```

Kode Program 2.1 Program *QueueArray*

```
package pekan4_2511532025;

public class QueueArrayDriver_2511532025 {

    public static void main(String[] args) {
        QueueArray_2511532025 queue_2025 = new
QueueArray_2511532025(1000);
        queue_2025.enqueue_2025(10);
        queue_2025.enqueue_2025(20);
        queue_2025.enqueue_2025(30);
        queue_2025.enqueue_2025(40);
        System.out.println("Item di depan " +
queue_2025.front_2025());
        System.out.println("Item paling belakang " +
queue_2025.rear_2025());
        System.out.println("tempilan queue");
        queue_2025.display_2025();
        System.out.println();
        System.out.println(queue_2025.dequeue_2025() + "
dihapus dari queue");
        System.out.println("item di depan: " +
queue_2025.front_2025());
        System.out.println("item di belakang: " +
queue_2025.rear_2025());
        System.out.println("tempilan queue setelah satu data
dihapus");
        queue_2025.display_2025();
    }
```

```
}
```

Kode Program 2.2 *Import QueueArrayDriver*

Kode di atas berfungsi untuk menunjukkan penggunaan struktur data *Queue* (antrian) dalam bahasa Java yang diimplementasikan menggunakan *array* dalam bentuk kelas. Program ini terdiri dari dua *class*, yaitu *class QueueArray*_2511532025 yang berfungsi sebagai pengelola data (menyimpan dan memanipulasi antrian), serta *class QueueArrayDriver*_2511532025 yang berfungsi sebagai *driver* atau program utama untuk menjalankan dan menguji *method* yang telah dibuat.

Program ini digunakan untuk menyimpan dan mengelola data dalam bentuk antrian dengan prinsip *FIFO* (*First In First Out*). Beberapa operasi yang dapat dilakukan antara lain menambahkan elemen ke dalam antrian (*enqueue*), menghapus elemen dari antrian (*dequeue*), melihat elemen terdepan (*front*), melihat elemen terakhir (*rear*), serta menampilkan seluruh isi antrian menggunakan *method display*.

Melalui program ini, dapat dipahami bagaimana cara kerja *Queue* menggunakan *array*, termasuk penggunaan konsep *circular queue* yang memungkinkan pemanfaatan *array* secara lebih efisien karena indeks dapat berputar kembali ke awal ketika mencapai batas maksimum.

2.1.1 Langkah Kerja Program

1. Membuat *class QueueArray_2511532025*

```
public class QueueArray_2511532025 {
```

Kode Program 2.3 *Class QueueArray*

Class ini digunakan untuk merepresentasikan struktur data *Queue* menggunakan *array*.

2. Deklarasi *variabel*

```
int front_2025, rear_2025, size_2025;
int capacity_2025;
int array_2025[];
```

Kode Program 2.4 Deklarasi *Variabel Queue*

Variabel ini berfungsi untuk:

- *front_2025* → menunjuk elemen depan
- *rear_2025* → menunjuk elemen belakang
- *size_2025* → jumlah elemen dalam *queue*
- *capacity_2025* → kapasitas maksimum *queue*
- *array_2025[]* → tempat penyimpanan data

3. Konstruktor *Queue*

```
public QueueArray_2511532025(int capacity_2025) {
  this.capacity_2025 = capacity_2025;
  front_2025 = this.size_2025 = 0;
  rear_2025 = capacity_2025 -1;
  array_2025 = new int[this.capacity_2025];
}
```

Kode Program 2.5 Konstruktor *Queue*

Konstruktor digunakan untuk:

- Menentukan kapasitas *queue*
- Menginisialisasi *front = 0*
- Mengatur *rear = capacity - 1*
- Membuat *array* sebagai penyimpanan

4. *Method isFull()*

```
boolean isFull(QueueArray_2511532025 queue) {
  return (queue.size_2025 == queue.capacity_2025);
}
```

Kode Program 2.6 *Method isFull()*

Digunakan untuk mengecek apakah *queue* sudah penuh. Jika *size* == *capacity*, maka *queue* penuh.

5. *Method isEmpty()*

```
boolean isEmpty(QueueArray_2511532025 queue) {
  return (queue.size_2025 == 0);
}
```

Kode Program 2.7 *Method isEmpty()*

Digunakan untuk mengecek apakah *queue* kosong. Jika *size* == *0*, maka *queue* kosong.

6. *Method enqueue()*

```
void enqueue_2025(int item) {
  if (isFull(this))
        return;
  this.rear_2025 = (this.rear_2025 + 1) %
this.capacity_2025;
  this.array_2025[this.rear_2025] = item;
  this.size_2025 = this.size_2025 + 1;
  System.out.println(item + " enqueued to queue");
}
```

Kode Program 2.8 *Method Enqueue*

Digunakan untuk menambahkan elemen ke dalam *queue*.

- Posisi *rear* akan berpindah menggunakan operasi modulo:

  (this.rear_2025 + 1) % this.capacity_2025
- Elemen dimasukkan ke *array*
- *size* bertambah 1

7. *Method dequeue()*

```
int dequeue_2025() {
  if (isEmpty(this))
        return Integer.MIN_VALUE;
  int item = this.array_2025[this.front_2025];
```

```
  this.front_2025 = (this.front_2025 + 1) %
this.capacity_2025;
  this.size_2025 = this.size_2025 - 1;
  return item;
  }
```

Kode Program 2.9 *Method Dequeue*

Digunakan untuk menghapus elemen dari *queue*.

- Elemen diambil dari posisi *front*
- *front* maju satu langkah (*circular*)
- *size* berkurang 1

8. *Method front()*

```
int front_2025() {
  if (isEmpty(this))
        return Integer.MIN_VALUE;
  return this.array_2025[this.front_2025];
|}
```

Kode Program 2.10 *Method Front*

Mengembalikan elemen paling depan tanpa menghapusnya.

9. *Method rear()*

```
int rear_2025() {
  if (isEmpty(this))
        return Integer.MIN_VALUE;
  return this.array_2025[this.rear_2025];
}
```

Kode Program 2.11 *Method Rear*

Mengembalikan elemen paling belakang.

10. *Method display()*

```
//mencetak elemen antrian
void display_2025() {
  int i_2025;
  if (front_2025 == rear_2025) {
        System.out.printf("\nAntrian Kosong\n");
        return;
  }
```

Kode Program 2.12 *Method Display*

Digunakan untuk menampilkan seluruh isi *queue* dari depan ke belakang.

11. Membuat *class QueueArray_2511532025*

```
package pekan4_2511532025;
```

Kode Program 2.13 *Class QueueArrayDriver*

Class ini digunakan untuk menguji dan menjalankan operasi pada struktur data *Queue* yang telah dibuat pada *class QueueArray*_2511532025.

12. Membuat objek *Queue*

```
QueueArray_2511532025 queue_2025 = new
QueueArray_2511532025(1000);
```

Kode Program 2.14 Pembuatan Objek *Queue*

Baris ini digunakan untuk membuat objek *queue* dengan kapasitas maksimum 1000 elemen.

13. Menambahkan elemen ke dalam *queue (enqueue)*

```
queue_2025.enqueue_2025(10);
queue_2025.enqueue_2025(20);
queue_2025.enqueue_2025(30);
queue_2025.enqueue_2025(40);
```

Kode Program 2.15 Proses *Enqueue*

Data dimasukkan ke dalam *queue* secara berurutan, yaitu 10, 20, 30, dan 40.

14. Menampilkan elemen depan dan belakang

```
System.out.println("Item di depan " +
queue_2025.front_2025());
System.out.println("Item paling belakang " +
queue_2025.rear_2025());
```

Kode Program 2.16 Menampilkan *Front* dan *Rear*

- *front()* → menampilkan elemen pertama (terdepan)
- *rear()* → menampilkan elemen terakhir

15. Menampilkan isi *queue*

```
System.out.println("tempilan queue");
queue_2025.display
_2025();
```

Kode Program 2.17 Menampilkan Isi *Queue*

Menampilkan seluruh isi antrian dari depan ke belakang.

16. Menghapus elemen dari *queue* (*dequeue*)

```
System.out.println(queue_2025.dequeue_2025() + "
dihapus dari queue");
```

Kode Program 2.18 Proses *Dequeue*

Menghapus elemen pertama dari *queue* sesuai prinsip *FIFO*.

17. Menampilkan kondisi *queue* setelah penghapusan

```
System.out.println("item di depan: " +
queue_2025.front_2025());
System.out.println("item di belakang: " +
queue_2025.rear_2025());
```

Kode Program 2.19 Kondisi *Queue* Setelah *Dequeue*

Menampilkan elemen depan dan belakang setelah satu data dihapus.

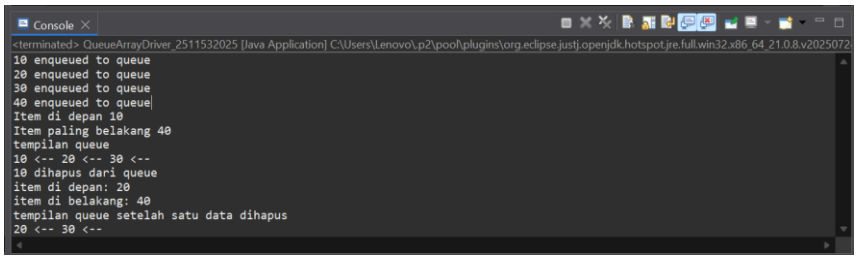
18. Menampilkan *queue* setelah perubahan

```
System.out.println("tampilan queue setelah satu data
dihapus");
queue_2025.display_2025();
```

Kode Program 2.20 Menampilkan *Queue* Setelah *Dequeue*

Menampilkan isi *queue* setelah dilakukan operasi *dequeue*.

2.1.2 Hasil Program



Gambar 2.1 Hasil Program *QueueArray*

Berdasarkan hasil eksekusi program, elemen dimasukkan ke dalam *queue* menggunakan *method enqueue*, sehingga data tersusun sesuai urutan masuk, misalnya:

10 <-- 20 <-- 30 <-- 40

Hal ini menunjukkan bahwa queue bekerja sesuai prinsip *FIFO* (*First In First Out*).

Ketika dilakukan operasi *dequeue*, elemen pertama yaitu 10 akan dihapus, sehingga isi *queue* menjadi:

20 <-- 30 <-- 40

Perubahan ini sesuai dengan teori *queue*, dimana elemen yang pertama masuk akan menjadi elemen pertama yang keluar.

Selain itu:

- *Method front()* menampilkan elemen terdepan
- *Method rear()* menampilkan elemen terakhir
- *size* berkurang setelah penghapusan

Program ini juga menunjukkan penggunaan *circular queue*, dimana indeks akan kembali ke awal ketika mencapai batas *array*, sehingga penggunaan memori menjadi lebih efisien.

2.2 Program *QueueLinkedList*

Kode Program yang dipraktikkan adalah sebagai berikut:

```java
package pekan4_2511532025;

import java.util.Queue;
import java.util.LinkedList;

public class QueueLinkedList_2511532025 {

   public static void main(String[] args) {
         Queue<Integer> q_2025 = new LinkedList<>();
         //tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
         for (int i_2025 = 0; i_2025 < 6; i_2025++)
               q_2025.add(i_2025);
         //menampilkan isi antrian.
         System.out.println("Elemen Antrian " + q_2025);
         //untuk menghapus kepala antrian.
         int hapus_2025 = q_2025.remove();
         System.out.println("Hapus elemen = " + hapus_2025);
         System.out.println(q_2025);
         //untuk melihat antrian terdepan
         int depan_2025 = q_2025.peek();
         System.out.println("Kepala antrian = " +
depan_2025);

         int banyak_2025 = q_2025.size();
         System.out.println("Size antrian = " + banyak_2025);
   }

}
```

Kode Program 2.21 Program *QueueLinkedList*

Kode program di atas berfungsi untuk mengimplementasikan struktur data *Queue* (antrian) menggunakan class bawaan Java, yaitu *LinkedList*. Pada program ini, *interface Queue* digunakan sebagai tipe data, sedangkan *LinkedList* sebagai implementasinya.

Program ini digunakan untuk menyimpan dan mengelola data dalam bentuk antrian dengan prinsip *FIFO* (*First In First Out*). Operasi yang dilakukan meliputi penambahan elemen (*enqueue*), penghapusan elemen (*dequeue*), melihat elemen terdepan (*peek*), serta mengetahui jumlah elemen dalam antrian (*size*).

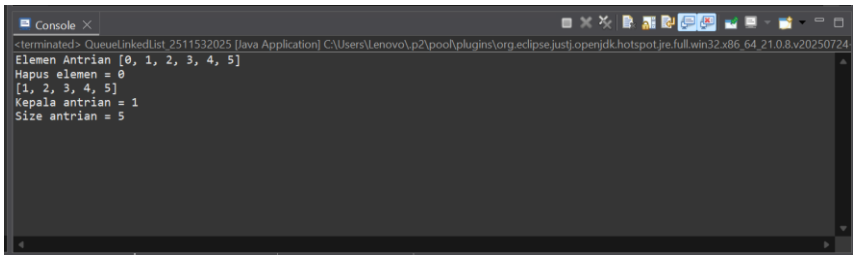
2.2.1 Langkah Kerja Program *QueueLinkedList*

Program diawali dengan mengimpor library *Queue* dan *LinkedList* dari *Java Collection Framework* yang digunakan untuk

mengimplementasikan struktur data antrian. Selanjutnya dibuat sebuah *class* bernama *QueueLinkedList_2511532025* yang berfungsi sebagai program utama. Di dalam *method main*, dibuat sebuah objek *queue* dengan tipe data Integer menggunakan implementasi *LinkedList*. Setelah itu, program menambahkan elemen ke dalam *queue* menggunakan perulangan, yaitu dari nilai 0 sampai 5, sehingga terbentuk antrian berisi beberapa data secara berurutan. Program kemudian menampilkan seluruh isi antrian untuk menunjukkan data yang telah dimasukkan.

Selanjutnya dilakukan operasi penghapusan elemen menggunakan *method remove()*, dimana elemen pertama dalam antrian akan dihapus sesuai dengan prinsip *FIFO* (*First In First Out*). Setelah penghapusan, isi *queue* kembali ditampilkan untuk melihat perubahan yang terjadi. Program juga menggunakan *method peek()* untuk melihat elemen yang berada di posisi terdepan tanpa menghapusnya. Terakhir, *method size()* digunakan untuk mengetahui jumlah elemen yang tersisa dalam *queue* setelah dilakukan operasi penghapusan. Dengan demikian, program ini menunjukkan cara kerja Queue menggunakan *LinkedList* secara lengkap, mulai dari penambahan, penghapusan, hingga pengamatan isi dan ukuran antrian.

2.2.2 Hasil Program



Gambar 2.2 Hasil Program *QueueLinkedList*

Berdasarkan hasil eksekusi program, *queue* awalnya berisi elemen:

[0, 1, 2, 3, 4, 5]

Setelah dilakukan operasi *remove()*, elemen pertama yaitu 0 dihapus, sehingga *queue* menjadi:

[1, 2, 3, 4, 5]

Kemudian, *method peek()* menampilkan elemen terdepan yaitu 1, tanpa menghapusnya dari *queue*. *Method size()* menunjukkan jumlah elemen dalam *queue* setelah penghapusan, yaitu sebanyak 5 elemen.

Hal ini membuktikan bahwa *Queue* menggunakan *LinkedList* bekerja sesuai prinsip *FIFO* (*First In First Out*).

2.3 Program *ReverseData*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan4_2511532025;

import java.util.LinkedList;
import java.util.Queue;
import java.util.Stack;

public class ReverseData_2511532025 {

   public static void main(String[] args) {
         Queue<Integer> q_2025 = new LinkedList<Integer>();
         q_2025.add(1);
         q_2025.add(2);
         q_2025.add(3); // [i, 2, 3]
         System.out.println("sebelum reverse = " + q_2025);
         Stack<Integer> s_2025 = new Stack<Integer>();
         while (!q_2025.isEmpty()) { // Q -> S
               s_2025.push(q_2025.remove());
         }
         while (!s_2025.isEmpty()) { // S -> Q
               q_2025.add(s_2025.pop());
         }
         System.out.println("sesudah reverse = " + q_2025);
// [3, 2, 1]
   }

}
```

Kode Program 2.22 Program *ReverseData*

Kode program di atas berfungsi untuk membalik urutan data dalam struktur data *Queue* menggunakan bantuan struktur data *Stack*. Program ini memanfaatkan dua struktur data sekaligus, yaitu *Queue* sebagai penyimpanan utama dan *Stack* sebagai alat bantu untuk melakukan proses pembalikan data.

Program bekerja dengan prinsip bahwa *Stack* menggunakan metode *LIFO* (*Last In First Out*), sedangkan *Queue* menggunakan metode *FIFO* (*First In First Out*)**.** Dengan memindahkan data dari *Queue* ke *Stack*, urutan data akan terbalik, kemudian saat dipindahkan kembali ke *Queue*, urutan yang sudah terbalik tersebut akan dipertahankan.

2.3.1 Langkah Kerja Program *ReverseData*

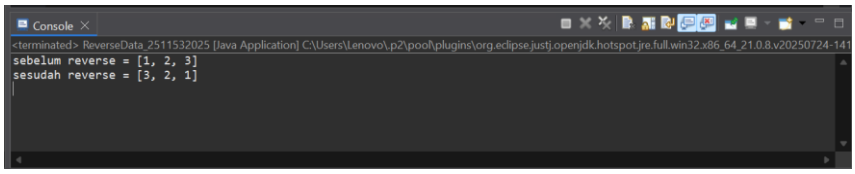
Program diawali dengan mengimpor *library Queue*, *LinkedList*, dan *Stack* dari *Java Collection Framework*. Selanjutnya dibuat sebuah *class* bernama *ReverseData_2511532025* yang berfungsi sebagai program utama. Di dalam *method main*, dibuat sebuah objek *queue* dengan tipe data *Integer*, kemudian ditambahkan beberapa elemen yaitu 1, 2, dan 3 ke dalam queue sehingga membentuk antrian awal.

Program kemudian menampilkan isi *queue* sebelum dilakukan proses pembalikan. Setelah itu, dibuat sebuah objek *stack* yang digunakan sebagai media bantu. Proses pembalikan dilakukan dalam dua tahap, yaitu memindahkan seluruh elemen dari *queue* ke *stack* menggunakan perulangan *while* hingga *queue* kosong. Karena *stack* bekerja dengan prinsip *LIFO*, urutan elemen menjadi terbalik saat dimasukkan ke dalam *stack*.

Selanjutnya, elemen dalam stack dipindahkan kembali ke queue menggunakan perulangan *while* hingga *stack* kosong. Proses ini menghasilkan *queue* dengan urutan elemen yang sudah

terbalik dari kondisi awal. Terakhir, program menampilkan isi *queue* setelah proses *reverse* untuk menunjukkan hasil akhir.

2.3.2 Hasil Program



Gambar 2.3 Hasil Program *ReverseData*

Berdasarkan hasil eksekusi program, *queue* awalnya berisi elemen:

[1, 2, 3]

Setelah dilakukan proses pemindahan ke *stack* dan dikembalikan ke *queue*, urutan elemen menjadi:

[3, 2, 1]

Hal ini menunjukkan bahwa proses *reverse* berhasil dilakukan. Perubahan urutan ini terjadi karena sifat *Stack* yang menggunakan prinsip *LIFO (Last In First Out)*, sehingga elemen terakhir yang masuk akan menjadi elemen pertama yang keluar. Dengan mengombinasikan *Queue* dan *Stack*, program ini berhasil membalik urutan data secara efektif.

2.4 Program *IterasiQueue*

Kode Program yang dipraktikkan adalah sebagai berikut:

```
package pekan4_2511532025;

import java.util.Iterator;
import java.util.Queue;
import java.util.LinkedList;

public class IterasiQueue_2511532025 {

   public static void main(String[] args) {
         Queue<String> q_2025 = new LinkedList<>();
```

```
        q_2025.add("Praktikum");
        q_2025.add("Struktur");
        q_2025.add("Data");
        q_2025.add("Dan");
        q_2025.add("Algoritma");
        Iterator<String> iterator = q_2025.iterator();
        while (iterator.hasNext()) {
              System.out.print(iterator.next() + " ");
        }
    }

}
```

Kode Program 2.22 Program *IterasiQueue*

Kode program di atas berfungsi untuk menampilkan elemen-elemen dalam struktur data *Queue* menggunakan metode *iterasi*. Program ini menggunakan *Iterator* untuk mengakses setiap elemen dalam *queue* secara berurutan tanpa menghapus data yang ada.

2.4.1 Langkah Kerja Program *IterasiQueue*

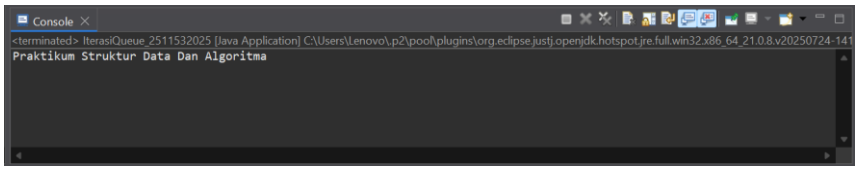
Program diawali dengan mengimpor *library Queue*, *LinkedList*, dan *Iterator* dari *Java Collection Framework*. Selanjutnya dibuat sebuah *class* bernama *IterasiQueue_2511532025* yang berfungsi sebagai program utama. Di dalam *method main*, dibuat sebuah objek *queue* dengan tipe data *String* menggunakan implementasi *LinkedList*.

Kemudian program menambahkan beberapa elemen berupa kata-kata, yaitu "Praktikum", "Struktur", "Data", "Dan", dan "Algoritma" ke dalam queue. Setelah itu, dibuat sebuah objek *Iterator* yang digunakan untuk menelusuri elemen dalam *queue*.

Proses iterasi dilakukan menggunakan perulangan *while* dengan kondisi *iterator.hasNext()*. Selama masih terdapat elemen berikutnya, *method iterator.next()* akan dipanggil untuk mengambil dan menampilkan setiap elemen secara berurutan.

Proses ini tidak mengubah atau menghapus isi *queue*, sehingga data tetap utuh setelah *iterasi* selesai.

2.4.2 Hasil Program



Gambar 2.4 Hasil Program *IterasiQueue*

Berdasarkan hasil eksekusi program, elemen-elemen dalam queue ditampilkan secara berurutan sebagai berikut:

Praktikum Struktur Data Dan Algoritma

Hasil ini menunjukkan bahwa iterasi berhasil dilakukan sesuai dengan urutan elemen dalam *queue*. Selain itu, penggunaan *Iterator* memungkinkan akses data tanpa mengubah struktur *queue*, sehingga aman digunakan untuk proses penelusuran data.

BAB III
KESIMPULAN

3.1 Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa *Queue* (antrian) merupakan salah satu struktur data penting dalam Java yang digunakan untuk mengelola data dengan prinsip *FIFO* (*First In First Out*), yaitu data yang pertama masuk akan menjadi data pertama yang keluar.

Melalui beberapa program yang telah dibuat, seperti *Queue* menggunakan *array* (*QueueArray*), *Queue* menggunakan *LinkedList*, pembalikan data menggunakan *Stack* (*ReverseData*), serta *iterasi Queue* menggunakan *Iterator*, dapat diketahui bahwa struktur data *Queue* memiliki berbagai cara implementasi dengan kelebihan masing-masing.

Implementasi *Queue* menggunakan array memungkinkan pengelolaan data secara terstruktur dengan memanfaatkan konsep *circular queue* sehingga penggunaan memori lebih efisien. Sementara itu, penggunaan *LinkedList* memberikan fleksibilitas karena tidak memiliki batas kapasitas tetap dan dapat menyesuaikan jumlah data secara dinamis.

Selain itu, penggunaan *Stack* dalam program *ReverseData* menunjukkan bahwa kombinasi beberapa struktur data dapat digunakan untuk menyelesaikan permasalahan tertentu, seperti membalik urutan data. Penggunaan *Iterator* juga mempermudah proses penelusuran data dalam *Queue* tanpa mengubah isi data tersebut.

Dengan demikian, praktikum ini membantu memahami konsep, cara kerja, serta implementasi berbagai struktur data dalam Java, serta meningkatkan kemampuan dalam membuat program untuk mengelola data secara lebih efisien dan terstruktur.

DAFTAR PUSTAKA

Wahyudi, S.T., M.T., *Struktur Data: Pekan 5 Queue*. Padang: Universitas Andalas, 2025.

Tugas Praktikum Pekan 4: Struktur Data *Queue*

1. Deskripsi Tugas

   Mahasiswa diminta untuk mengimplementasikan struktur data *Queue* (Antrian) menggunakan *array* dalam bahasa Java. Studi kasus yang digunakan adalah antrian pelanggan di loket pelayanan. Setiap pelanggan yang datang akan masuk ke dalam antrian dan akan dilayani berdasarkan prinsip *FIFO* (*First In First Out*).

   Program harus mampu:

   - Menambahkan pelanggan ke antrian (*enqueue*)
   - Menghapus pelanggan dari antrian (*dequeue*)
   - Menampilkan isi antrian
   - Mengecek kondisi antrian (kosong atau penuh)

2. Kode Program

```java
package pekan4_2511532025;

import java.util.Scanner;

public class AntrianLoket_2511532025 {

    // deklarasi variabel
    private String queue_2025[];
    private int front_2025;
    private int rear_2025;
    private int max_2025;

    // konstruktor
    public AntrianLoket_2511532025(int kapasitas) {
        max_2025 = kapasitas;
        queue_2025 = new String[max_2025];
        front_2025 = -1;
        rear_2025 = -1;
    }

    // cek kosong
    public boolean isEmpty_2025() {
        return (front_2025 == -1);
    }

    // cek penuh
    public boolean isFull_2025() {
        return (rear_2025 == max_2025 - 1);
    }

    // enqueue
    public void enqueue_2025(String data) {
```

```java
        if (isFull_2025()) {
            System.out.println("Antrian penuh!");
        } else {
            if (isEmpty_2025()) {
                front_2025 = 0;
            }
            rear_2025++;
            queue_2025[rear_2025] = data;
            System.out.println("Data berhasil ditambahkan ke
antrian");
        }
    }

    // dequeue
    public void dequeue_2025() {
        if (isEmpty_2025()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println(queue_2025[front_2025] + " telah
dilayani");

            if (front_2025 == rear_2025) {
                front_2025 = -1;
                rear_2025 = -1;
            } else {
                front_2025++;
            }
        }
    }

    // display
    public void display_2025() {
        if (isEmpty_2025()) {
            System.out.println("Antrian kosong!");
        } else {
            System.out.println("Isi antrian:");
            int no = 1;
            for (int i = front_2025; i <= rear_2025; i++) {
                System.out.println(no + ". " + queue_2025[i]);
                no++;
            }
        }
    }

    // reverse
    public void reverse_2025() {
        if (isEmpty_2025()) {
            System.out.println("Antrian kosong!");
        } else {
            int i = front_2025;
            int j = rear_2025;

            while (i < j) {
                String temp = queue_2025[i];
                queue_2025[i] = queue_2025[j];
                queue_2025[j] = temp;
                i++;
```

```
                j--;
            }
            System.out.println("Antrian berhasil dibalik");
        }
    }

    // main program
    public static void main(String[] args) {
        Scanner input_2025 = new Scanner(System.in);
        AntrianLoket_2511532025 antrian_2025 = new
AntrianLoket_2511532025(10);

        int pilihan_2025;

        do {
            System.out.println("\n=== PROGRAM ANTRIAN LOKET
===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Tampilkan Antrian");
            System.out.println("4. Reverse");
            System.out.println("5. Keluar");
            System.out.print("Pilih menu: ");
            pilihan_2025 = input_2025.nextInt();
            input_2025.nextLine();

            switch (pilihan_2025) {
                case 1:
                    System.out.print("Masukkan nama pelanggan:
");
                    String nama_2025 = input_2025.nextLine();
                    antrian_2025.enqueue_2025(nama_2025);
                    break;

                case 2:
                    antrian_2025.dequeue_2025();
                    break;

                case 3:
                    antrian_2025.display_2025();
                    break;

                case 4:
                    antrian_2025.reverse_2025();
                    antrian_2025.display_2025();
                    break;

                case 5:
                    System.out.println("Program selesai");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
            }

        } while (pilihan_2025 != 5);
```

```
        input_2025.close();
    }

}
```

3. Tampilan Output Program

```
Console
<terminated> AntrianLoket_2511532025 [Java Application] C:\Users\Lenovo\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-14

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: ariba
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 1
Masukkan nama pelanggan: tsabita
Data berhasil ditambahkan ke antrian

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. ariba
2. tsabita
```

```
Console
<terminated> AntrianLoket_2511532025 [Java Application] C:\Users\Lenovo\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.8.v20250724-14
=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 4
Antrian berhasil dibalik
Isi antrian:
1. tsabita
2. ariba

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 2
tsabita telah dilayani

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 3
Isi antrian:
1. ariba

=== PROGRAM ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Tampilkan Antrian
4. Reverse
5. Keluar
Pilih menu: 5
Program selesai
```